CHAPTER 12: OBSERVABILITY AT SCALE

Summary: *Observability questions punish one specific confusion: mistaking something that reads telemetry for something that produces it. Dashboards, alerts and runbooks every one of them read numbers somebody else had to emit first, so a plan assembled from those three instruments nothing at all. This chapter separates the decisions that change what the code emits from the ones that only change what you do with it, covers the failures that arrive looking like successes, and sets out what is switched off until you switch it on.*

Two Decisions Determine What Gets Written

When a scenario asks which steps are needed to instrument a system, the discriminating property is sharp: does this **step change what the instrumentation code actually writes**?

Exactly two do.

The **reliability targets**. What the system is supposed to achieve determines what you have to be able to measure. Without a target there is no way to know whether a number is a problem.

The **definition of signals and slices**. Which measurements get emitted, and which dimensions they carry, so results can be broken down by version, by customer segment, by request type.

Everything else on the usual list reads data rather than creating any. A **threshold needs numbers to sit above**. A **chart needs something to plot**. A **runbook needs a condition somebody already detected**. All three sit downstream, and none of them can be built usefully until the emission decisions are settled.

So a scenario asking for two steps and offering five is testing exactly this. Reach for a third and you have picked something that **reads the data** rather than something that **decides what the data** is.

The Failure Taxonomy

The reason observability for these systems needs its own thinking is that the failure modes are unfamiliar to existing monitoring. Name the set you are instrumenting

for: the **failure taxonomy** is the list of distinct ways this system can be wrong, and each entry needs its own detection.

Failure	What it looks like to existing monitoring
A confident wrong answer	A success. 200, normal latency, well-formed output
A refusal	Also a success, on a normal response stream
A truncated answer	A success with a stop reason nobody read
Silent capability degradation	Nothing at all, unless a usage field is inspected
Retrieval returning stale material	A success with citations attached
A tool call that never completed	A response with one fewer result block than calls

Look at the right-hand column. **Almost every entry reads as success**. That is the whole problem, and it is why "we have monitoring" is never an answer to "how would you find out?"

Always-On Signal, Selectively Captured Payloads

Cost pressure on telemetry produces one good design and several bad ones, and they are easy to tell apart once you know the split.

The good design keeps **two tiers**: a **light structured record written for every session** without exception, and the **heavy material kept only where it will be read**. The light tier is identifiers, timing, outcome, version, dimensions. The heavy tier is full prompts, full responses, retrieved documents.

The result is that you can still **locate and reassemble any session** at all, while paying to keep the **bulky artifacts only where they earn** it.

The bad designs all save money by **deleting signal**: sampling sessions, dropping logs for successful requests, retaining only errors. Each cuts storage. Each also leaves you unable to find the failure afterwards, because the thing you are hunting looked like a success at the moment the sampling decision was taken. You **cannot sample on a property you can only determine afterwards**.

The Attributes Whose Absence You Cannot See

There is a class of instrumentation defect defined by what is **missing** rather than by what is wrong, and it is the hardest kind to spot in an option list because nothing in the emitted data looks incorrect.

Two attributes matter more than any others, because they are the joins.

The **version identifier**. Without it, telemetry cannot be attributed to a version of the system, so no before-and-after comparison is possible and no regression can be located in time.

The **correlation identifier**. Without it, the records belonging to one request cannot be reassembled into a single request, so you have a pile of true facts about steps and no way to see the transaction they belong to.

Both are invisible while things work. Both get discovered during the first incident, which is the worst possible moment for a room of humans to learn that their data cannot answer the question.

The **named anti-pattern is instrumentation with no join keys**. Every individual metric is correct and the set of them cannot be assembled into a story.

Where Failures Hide

Three documented behaviours put failures somewhere your error handling is not looking.

Errors arrive inside the event stream. Streaming responses use server-sent events, and the API may send errors within the stream itself. The documented example is an overloaded error, which in a non-streaming context would normally correspond

to an HTTP 529.[1] A streaming client that only handles **transport-level failures** will **miss errors delivered as events**.

A **refusal fires a refusal event, not an error event**. The monitoring reference documents a separate refusal event alongside the API error event, and explains why: refusals arrive on a successful response stream, so the error event does not fire for them.[2] If your alerting is wired to error events, refusals are invisible.

Cache diagnostics report only the earliest divergence. The feature compares a request against a previous response id and reports the first point of difference among the model, the system prompt, the tools and the message history. Because only the earliest is reported, **later divergences may be hidden behind** it.[3] So a clean-looking result after one fix does not mean there was only one cause. Fix and re-run, rather than fix and assume.

Telemetry Is Off Until You Turn It On, Twice

Two documented defaults will produce a scenario in which somebody believed they had observability and had none.

Distributed tracing is off by default and requires **two environment variables** to enable.[2] One is not enough.

Agent SDK telemetry is off until an **enabling environment variable** is set and **at least one exporter** is chosen.[4] Again, both conditions. And the architecture is worth knowing because it explains where to look when nothing appears: the **SDK produces no telemetry of its own**. It passes configuration through to the instrumented command-line process, and that process records spans around each model request and tool execution and exports directly to the configured collector.

The span hierarchy is documented and worth carrying, because it tells you what a well-formed trace should look like:[2]

- **Each user prompt opens a root interaction span**.
- **API calls, tool calls and hook executions are recorded as children** of it.
- A **subagent's spans nest under the parent tool span**, so delegated work stays attached to the call that delegated it.

If a trace does not have that shape, something is not instrumented rather than something is not happening.

One tuning control comes with a stated trade-off: environment variables control metrics cardinality, and **lower cardinality** generally means **better performance and lower storage costs**, at the price of **less granular data** for analysis.[2] That is a genuine trade rather than a free optimisation, and the dimensions you drop are exactly the ones you will want during an incident.

Choose the Bucket Width for the Question

The organisation usage report returns token consumption grouped and filtered by model, workspace, API key, service tier, context window, inference region and speed, at three bucket widths, each documented for a purpose:[5]

Width	Documented use
One minute	Real-time monitoring
One hour	Daily patterns
One day	Weekly and monthly reports

A separate cost report endpoint returns service-level cost breakdowns in US dollars.[5]

Which brings up the distinction that decides the remaining items in this area. A measurement taken **inside the pipeline** while the request is being served **can move before any human feels the degradation**. A number read off a billing cycle or a reporting cycle is a downstream consequence: true, useful for accounting, and always describing something that already happened to somebody.

So when a scenario asks for an early signal, look for whatever is measured **inside the request path, before the answer reaches a human**. Cost reports and usage summaries are not early warnings, whatever their bucket width.

A Runbook Holds the Response, Not the Detection

Runbooks appear twice in this chapter as things that **consume** instrumentation, and that stays true. It leaves a question open that scenarios ask directly: once the numbers exist, what actually goes inside one.

Five things, and they divide into what happened and what you do about it.

The **alerts that fire in normal operation**, named individually, so every page a person receives has a document behind it rather than a folk memory.

The **triage steps for each one**, in order, so the first ten minutes are not being invented at three in the morning by whoever is on call.

The **escalation path**: who gets woken, at what point, and through which mechanism.

The **rollback procedure**, written to be executed by somebody who did not ship the change and does not know what it contained.

Pointers to the dashboards and logs carrying the evidence, so the reader is not composing a query while the incident runs.

What is **not** in a runbook is anything that decides **whether to fire at all**. Thresholds, service-level objectives and the slices they are measured over are the **inputs that produce the alert**. The runbook begins at the moment the alert has already arrived, which is why it can only be written after those numbers exist.

How Wrong Answers Are Built on Observability Questions

- A **consumer of telemetry offered as a source** of it. The option proposes dashboards, alert thresholds or runbooks as instrumentation steps. The test: could this be **built before any data exists**? If yes, it **consumes rather than produces**.

- **Saving money by deleting signal**. The option samples sessions or retains only errors. The test: would this have **captured the failure you are trying to find**, given that it **looked like a success at the time**?

- A **downstream number offered as an early signal**. The test: is this generated **inside the request path**, or read from a **billing or reporting cycle afterwards**?
- **Error handling wired to the wrong channel**. The test: does this failure arrive as an **HTTP error**, or on a **successful response**?
- An **emitted-data defect assumed where the defect is missing data**. The test: is anything **wrong with what was written**, or is something **simply absent**?

When the scenario says	The answer is usually	The trap is usually
Which steps are needed to instrument a system, two wanted	Reliability targets, and the definition of signals and slices	Thresholds, charts or runbooks, every one of which needs numbers first
Telemetry costs too much to keep	A light record for every session, heavy payloads kept selectively	Sampling sessions or keeping only errors, which destroys findability
Failures cannot be traced to a release, or steps cannot be joined	The instrumentation is not emitting version and correlation identifiers	Looking for something wrong in what it does emit
An early warning of quality degradation is wanted	Something measured inside the request path as it is served	Cost reports, usage summaries or anything read on a billing cycle
A streaming client sees no errors	Errors can arrive as events inside the stream	Handling only transport-level failures

When the scenario says	The answer is usually	The trap is usually
Refusals are not showing up in alerting	They fire a refusal event; the error event does not fire	Wiring alerts to error events only
A cache miss persists after one fix	Only the earliest divergence is reported; re-run and check again	Assuming one cause because one was reported
Tracing produces nothing	Two environment variables are required, and the SDK emits nothing itself	Assuming a single setting enables it
Cardinality is reduced to save storage	Accept that analysis granularity falls with it	Treating it as a free optimisation
Real-time monitoring is needed	One-minute buckets	One-day buckets, which are documented for monthly reporting
A runbook has to be filled in	Named alerts, triage steps in order, escalation path, rollback procedure, pointers to dashboards and logs	Thresholds and objectives, which are the inputs that produce the alert

The habit worth carrying: for every failure mode, name the **field that would change when it happens**. If you cannot name one, that failure is currently undetectable no matter how many dashboards exist.

What to Remember

- Only **two decisions** change what instrumentation code writes: the **reliability targets**, and the **definition of the signals and their slices**.

Thresholds, charts and runbooks all **read numbers that something else had to emit first**, so choosing any of them as an instrumentation step is **choosing a consumer instead of a determinant**.

- Almost every failure mode in these systems **reads as a success to existing monitoring**: confident wrong answers, refusals, truncation, silent degradation, stale retrieval and unfinished tool calls all arrive on **healthy-looking responses**.
- Keep **two tiers**: a **light structured record for every session** without exception, and **heavy artifacts only where somebody reads** them, so **any session can still be located and reassembled**. Sampling sessions or keeping only errors **throws away the very records you will need**, because the failure **looked like a success** at the moment that decision was taken.
- The two attributes whose absence breaks everything are the **version identifier** and the **correlation identifier**: without them telemetry **cannot be attributed to a release** and **cannot be reassembled into a single request**. Both defects are **invisible until the first incident**.
- **Errors can arrive inside a streaming event stream**, with an overloaded error given as the example, corresponding to an **HTTP 529** in a non-streaming context. A client handling only transport failures will **miss** them.
- Refusals fire a **dedicated refusal event** and do **not fire the API error event**, because they arrive on a **successful response stream**.
- Cache diagnostics report **only the earliest divergence**, so **later ones can hide behind** it. **Fix and re-run** rather than fix and assume.
- **Distributed tracing is off by default** and needs **two environment variables**. Agent SDK telemetry needs an **enabling variable** and at **least one exporter**. The **SDK emits no telemetry itself**; it passes configuration to the instrumented command-line process, which records spans around each model request and tool execution and exports to the collector directly.

- The span hierarchy is: a **root interaction span per user prompt**, with **API calls, tool calls and hook executions as children**, and a **subagent's spans nested beneath the parent tool span**. A trace of a different shape means **something is uninstrumented**.
- **Lower metrics cardinality** means **better performance and cheaper storage** in exchange for **less granular analysis**. The dimensions you drop are the **ones an incident will want**.
- Usage reporting groups and filters by **model**, **workspace**, **API key**, **service tier**, **context window**, **inference region** and **speed**, at **one-minute buckets** for real-time monitoring, **one-hour** for daily patterns and **one-day** for weekly and monthly reports, with a separate endpoint returning **service-level cost breakdowns in US dollars**.
- An **early signal** is measured **inside the request path** while the request is being served, **before the answer reaches a human**. Anything read from a billing or reporting cycle is a **downstream consequence** describing **something that already happened**.
- A **runbook holds the response, not the detection**: **named alerts**, **triage steps in order**, the **escalation path**, the **rollback procedure**, and **pointers to the dashboards and logs**. **Thresholds and service-level objectives are inputs that produce the alert**, so they **sit outside** the document that answers it.